

USER MANUAL

Unitree L1 User Manual

Simple Docker operation
Read, map, view, and record ROS 2 data

Simplified OctoMap edition
Prepared 3 August 2026
Validated project baseline simplified OctoMap working-tree baseline
Audience: Operators and first-time ROS 2 users

DOCUMENT CONTROL

This document is generated from `user-manual.md`. Commands and paths describe the repository state identified above.

Operate the powered LiDAR only after reading the safety section.

This manual is the complete operator procedure for reading a Unitree L1 in ROS 2 Humble, viewing its point cloud and stationary OctoMap in RViz2, and recording the point cloud and IMU with rosbag2. It deliberately uses ordinary Docker Compose and ROS 2 commands. There are no project wrapper scripts, custom processing nodes, or hidden services.

Run Docker commands on the Ubuntu host from:

```
/home/isr/unitree_l1_project
```

Run ROS 2 commands only after entering the `ros` container. The two prompts have different responsibilities:

Prompt or context	Where it runs	What belongs there
Host project directory	Ubuntu host	<code>docker compose</code> and host-device checks
<code>ros@...:/workspace\$</code>	Docker container	<code>ros2 launch</code> , topic checks, and rosbag2

What the operator starts

The project has one Docker Compose service named `ros`. Starting that service creates one long-running container and exposes the selected serial device, X11 display, and graphics device to it. Starting the container alone does not open the LiDAR and does not start a ROS node.

The live launch in Terminal A starts exactly these application processes:

Process	Condition	Purpose
<code>ros2 launch</code>	Always	Supervises the launch description
<code>unitree_lidar_ros2_node</code>	Always	Opens <code>/dev/unitree_lidar</code> and publishes sensor data
<code>ros2 topic hz /unilidar/cloud</code>	<code>monitor:=true</code>	Prints the received cloud rate
<code>octomap_server_node</code>	Combined OctoMap launch	Builds occupied and free voxels from the cloud
<code>rviz2</code>	<code>rviz:=true</code>	Displays <code>/unilidar/cloud</code> and, in map mode, occupied voxels

The record command in Terminal B starts one additional process, `ros2 bag record`. It subscribes only to `/unilidar/cloud` and `/unilidar/imu` and writes a bag beneath `/workspace/bags/`.

Docker Compose starts `/sbin/docker-init` as PID 1. The ROS base-image entrypoint sources ROS 2 and then executes `sleep infinity` as the init process's idle child. Docker init forwards stop signals and reaps children. Neither idle process reads the LiDAR or publishes ROS data. ROS 2 command-line use may start the normal ROS 2 daemon inside the container.

The host remains responsible for Docker, the physical tty device, the X11 display, graphics devices, and storage. ROS 2 Humble and RViz2 are not run directly on the Ubuntu 24.04 host.

SAFETY / WARNING

Secure the L1 before applying power. Keep hands, loose clothing, cables, and tools outside its moving and optical area. USB carries serial data; it is not the L1 power supply. Use the correct separate Unitree power supply and adapter.

1. Configure and build once

1.1 Normal defaults

Open a graphical host terminal and move to the project:

```
cd /home/isr/unitree_l1_project
```

The Compose file contains defaults for the validated computer. Most runs need no environment overrides. The important host values are:

Variable	Default or requirement	Meaning
HOST_UID, HOST_GID	1000, 1000	Ownership used to build the image
LIDAR_DEVICE	/dev/ttyUSB0	Host tty exposed as /dev/unitree_lidar
LIDAR_GID	20	Numeric group allowed to read and write the tty
DISPLAY	Required from host	Current X11 display
XAUTHORITY	Required host path	X11 cookie mounted read-only
VIDEO_GID	44	Numeric video-device group
RENDER_GID	992	Numeric render-device group

Do not set values merely because they exist. Override only a value that is different on the current host.

1.2 Optional host overrides

First inspect the current session:

```
id
printf 'DISPLAY=%s\nXAUTHORITY=%s\n' "$DISPLAY" "$XAUTHORITY"
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
ls -l /dev/dri/card0 /dev/dri/renderD128
```

The tty lines are meaningful only when the USB adapter is already connected. Otherwise determine the tty after Section 2.1. The device and device-group values affect container creation, not the image build.

If the L1 adapter appears at a different tty, set the real path and derive its group from the device itself:

```
export LIDAR_DEVICE=/dev/ttyUSB1
export LIDAR_GID="$(stat -Lc %g "$LIDAR_DEVICE")"
```

If host ownership or graphics groups differ from the defaults, derive them instead of guessing:

```
export HOST_UID="$(id -u)"
export HOST_GID="$(id -g)"
export VIDEO_GID="$(stat -Lc %g /dev/dri/card0)"
export RENDER_GID="$(stat -Lc %g /dev/dri/renderD128)"
```

If XAUTHORITY is empty but the graphical session uses the GDM cookie, set:

```
export XAUTHORITY="/run/user/$(id -u)/gdm/Xauthority"
```

Keep these exports in the host terminal used for `docker compose build` and `docker compose up`. ROS 2 uses its normal domain 0; this simple project does not add a domain override. Recreate the container after changing a device, group, or X11 value:

```
docker compose down
docker compose up -d
```

SAFETY / WARNING

Do not solve access problems with `privileged: true`, `chmod 777`, or a global `xhost +`. Correct the selected device, numeric group, display, or cookie instead.

1.3 Build the image

Build once after cloning the project and again only when the Dockerfile or ROS 2 source changes:

```
docker compose build
```

The build creates the ROS 2 Humble image and compiles the project packages. It does not start the LiDAR. A successful build returns to the host prompt without an error.

ENGINEERING NOTE

Daily operation does not require another build. Changing only a bag name or launch argument does not require rebuilding.

2. Connect the L1 and start Docker

2.1 Hardware connection

Complete the following with the L1 secured:

1. Confirm that the separate L1 power supply is off.
2. Connect the L1 to the correct Unitree serial adapter.
3. Connect the adapter to the computer's USB port.
4. Connect the correct separate power supply to the L1.
5. Clear the full operating area, then apply L1 power.
6. Wait for the mechanism and USB tty to stabilize.

USB and the separate supply serve different purposes:

```
L1 -> Unitree adapter -> host USB: serial data
L1 -> correct external supply: sensor power
```

Check that a tty exists on the host:

```
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

If the path differs from the Compose default, set `LIDAR_DEVICE` and `LIDAR_GID` as shown in Section 1.2 before starting the container.

2.2 Start the one project container

From the project root:

```
docker compose up -d
docker compose ps
```

The `ros` service should report Up. At this point the container is ready, but the driver, cloud-rate display, RViz2, and recorder are not yet running.

Check the mapped device without opening it:

```
docker compose exec ros bash -lc \
'id; ls -l /dev/unitree_lidar; test -r /dev/unitree_lidar; test -w /dev/unitree_lidar'
```

No output from the two `test` commands means both checks passed. A non-zero error or `Permission denied` must be corrected before launch.

Check the GUI boundary:

```
docker compose exec ros bash -lc \
'printf "DISPLAY=%s XAUTHORITY=%s\n" "$DISPLAY" "$XAUTHORITY"; test -r "$XAUTHORITY"'
```

The display values must be non-empty and the authorization file readable.

EXPECTED RESULT

One container named by the `ros` service is up, `/dev/unitree_lidar` is a readable and writable character device, and the container can read its X11 cookie.

3. Launch the LiDAR, OctoMap, and RViz2

3.1 Terminal A

Open Terminal A on the host, enter the project, and open a login shell in the running container:

```
cd /home/isr/unitree_l1_project
docker compose exec ros bash -l
```

The prompt changes to the container. Secure the L1 body so it cannot translate or rotate, then start the exact stationary mapping launch:

```
ros2 launch l1_octomap_bringup unitree_l1_octomap.launch.py \
  port:=/dev/unitree_lidar monitor:=true rviz:=true
```

Leave this command and Terminal A running. Do not start a second copy of the driver. The launch now owns the serial port.

3.2 Expected terminal and RViz2 behavior

Terminal A should show that the driver, cloud-rate command, OctoMap server, and RViz2 processes started. The driver should not repeatedly report that it cannot open the port. After clouds arrive, the standard `ros2 topic hz` command periodically prints the measured `/unilidar/cloud` rate.

RViz2 opens with the project profile:

```
Fixed frame: unilidar_lidar
PointCloud2 topic: /unilidar/cloud
MarkerArray topic: /occupied_cells_vis_array
```

With the L1 operating and an object in view, RViz2 should show a continuously updating three-dimensional point cloud and occupied OctoMap voxels around the sensor frame. The display contains a grid and axes. Rotate the camera with the mouse to distinguish a true three-dimensional map from a flat viewing angle.

EXPECTED RESULT

RViz2 stays open without an X11 error, displays fresh points on `/unilidar/cloud`, and begins showing occupied voxels after several scans. A running process with an empty view is not a successful sensor check.

3.3 What the launch arguments mean

Argument	Effect
<code>port:=/dev/unitree_lidar</code>	Uses the single tty mapped by Compose
<code>monitor:=true</code>	Runs <code>ros2 topic hz /unilidar/cloud</code>
<code>rviz:=true</code>	Starts RViz2 with the combined cloud and map profile
<code>resolution:=0.10</code>	Optional override for OctoMap voxel size in metres
<code>max_range:=15.0</code>	Optional override for maximum inserted range in metres

The driver publishes:

Topic	ROS 2 type	Meaning
<code>/unilidar/cloud</code>	<code>sensor_msgs/msg/PointCloud2</code>	L1 point cloud
<code>/unilidar/imu</code>	<code>sensor_msgs/msg/Imu</code>	L1 inertial measurement

Both topics are raw sensor data and are recorded in the normal bag procedure. The OctoMap launch additionally publishes `/occupied_cells_vis_array` and `/octomap_binary`. The map fixed frame equals the cloud frame, so no TF process is started. Keep the sensor body stationary: OctoMap is mapping, not SLAM.

For raw LiDAR visualization without OctoMap, use this alternative after stopping the combined launch:

```
ros2 launch l1_bringup unitree_l1.launch.py \
  port:=/dev/unitree_lidar monitor:=true rviz:=true
```

Never run both launches together because each one starts the LiDAR driver.

4. Confirm topics and record data

4.1 Terminal B

While Terminal A and RViz2 remain running, open a second host terminal:

```
cd /home/isr/unitree_l1_project
docker compose exec ros bash -l
```

This creates a second shell in the same container. It does not create a second container or driver.

List the ROS graph:

```
ros2 node list
ros2 topic list
```

The node list must include the driver, OctoMap server, and RViz2. It may also show generated ROS 2 command-line nodes. The topic list should include:

```
/unilidar/cloud
/unilidar/imu
/occupied_cells_vis_array
/octomap_binary
```

Inspect the publisher and subscriber counts:

```
ros2 topic info /unilidar/cloud
ros2 topic info /unilidar/imu
```

Each sensor topic must have one publisher. Confirm that one real message arrives on each topic without printing the large point array:

```
ros2 topic echo /unilidar/cloud --once --field header
ros2 topic echo /unilidar/imu --once --field header
```

Each command should print a stamped header and return to the prompt. If a command waits indefinitely, no matching message has arrived; do not begin an important recording.

Confirm that OctoMap receives the cloud and has occupied output:

```
ros2 node info /octomap_server
ros2 topic echo /octomap_point_cloud_centers --once --field width
ros2 topic echo /octomap_binary --once --no-arr
```

/octomap_server must list /unilidar/cloud as a subscription. After the sensor sees surfaces within 15 m, the occupied-center width must be greater than zero and the binary map must identify OcTree with a non-empty data field. Keep the L1 body fixed throughout the mapping session.

4.2 Start the recorder

Use a new output name for every recording:

```
ros2 bag record -o /workspace/bags/l1_run_01 \
  /unilidar/cloud /unilidar/imu
```

Leave Terminal B running while data is needed. The recorder subscribes to the two named topics and writes the bag into the host-visible bags/ directory.

The output path is a directory, not a single file. It contains a metadata.yaml file and one or more rosbag storage files.

SAFETY / WARNING

rosbag2 refuses to use an output directory that already exists. Never reuse l1_run_01. Choose l1_run_02, l1_lab_01, or another unique name. Do not delete an old bag merely to make a command succeed.

4.3 Recording checks

The recorder should report that it is listening or recording. During the recording:

- Terminal A remains on the live launch.
- RViz2 continues to update.
- Terminal B remains on `ros2 bag record`.
- The L1 and its cables remain secure.
- The host must retain enough free storage.

In a third host terminal, an optional size check is:

```
cd /home/isr/unitree_l1_project
du -sh bags/l1_run_01
```

Increasing size is useful evidence, but the final bag information after a clean stop is the authoritative check.

5. Stop cleanly and inspect the bag

The shutdown order protects the bag metadata and gives each process a clean ROS shutdown.

5.1 Stop the recorder first

Select Terminal B, where `ros2 bag record` is in the foreground, and press `Ctrl-C` once. Wait until the recorder finishes closing its storage and the container shell prompt returns.

Do not close the terminal window and do not run `docker compose down` while the recorder is still active.

Inspect the completed bag in the same container shell:

```
ros2 bag info /workspace/bags/l1_run_01
```

Check that:

- Duration is greater than zero.
- The total message count is greater than zero.
- `/unilidar/cloud` is present with type `sensor_msgs/msg/PointCloud2`.
- `/unilidar/imu` is present with type `sensor_msgs/msg/Imu`.
- Each recorded topic has a non-zero message count.

EXPECTED RESULT

The bag reports both requested sensor topics, non-zero counts, and a non-zero duration.

5.2 Stop the live launch second

Select Terminal A and press `Ctrl-C` once. Wait for `ros2 launch` to stop the driver, cloud-rate command, OctoMap server, and RViz2 and return to the container shell prompt.

Exit both container shells:

```
exit
```

Back on the host, remove the project container and its Compose network:

```
cd /home/isr/unitree_l1_project
docker compose down
```

The image and recorded bag remain. `docker compose down` removes the running container; it does not delete `/home/isr/unitree_l1_project/bags/l1_run_01`. After all software has stopped, remove L1 power before changing wiring.

5.3 Play back a bag

Playback does not need the live driver. With this deliberately simple single-service Compose design, the service still declares the tty mapping. Therefore the configured host tty must exist when `docker compose up -d` creates the playback container, even though `ros2 bag play` will not read it. Leaving the USB adapter connected is sufficient; do not start the live driver.

Start the container and open Terminal A:

```
cd /home/isr/unitree_l1_project
docker compose up -d
docker compose exec ros bash -l
rviz2 -d "$(ros2 pkg prefix --share l1_bringup)/config/unitree_l1.rviz"
```

Leave RViz2 running. In Terminal B:

```
cd /home/isr/unitree_l1_project
docker compose exec ros bash -l
ros2 bag play /workspace/bags/l1_run_01
```

RViz2 should display the recorded point cloud. When playback ends, stop RViz2 with `Ctrl-C`, exit both shells, and run `docker compose down` on the host.

SAFETY / WARNING

Do not run `unitree_l1.launch.py` during playback. A live driver and a bag would publish the same topic names and mix live and recorded data.

6. Common problems and direct checks

Compose cannot find the project

Symptom: `docker compose` reports that no configuration file exists.

Correction:

```
cd /home/isr/unitree_l1_project
docker compose config --quiet
```

Run every host Compose command from that directory.

Docker access is denied

Symptom: the Docker client cannot connect to the daemon socket.

Check:

```
docker version
docker compose version
```

This is a host Docker installation or account problem, not a ROS problem. Correct Docker access before continuing; do not use a root container as a substitute.

The `ros` service is not running

Symptom: `docker compose exec ros ...` says the service is not running.

Check and start it:

```
docker compose ps
docker compose up -d
docker compose logs ros
```

The host `tty` does not exist

Symptom: Compose reports a missing device or the container has no `/dev/unitree_lidar`.

Check the adapter on the host:

```
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
ls -l /dev/serial/by-id/ 2>/dev/null
```

Reconnect the Unitree adapter, wait for enumeration, then set the correct `LIDAR_DEVICE`. A separately powered sensor can still have a USB cabling or adapter problem.

The mapped `tty` says `Permission denied`

Derive the group from the selected host device:

```
export LIDAR_DEVICE="{LIDAR_DEVICE;--/dev/ttyUSB0}"
export LIDAR_GID="$(stat -Lc %g "$LIDAR_DEVICE")"
docker compose down
docker compose up -d
docker compose exec ros bash -lc 'id; ls -l /dev/unitree_lidar'
```

The numeric group must appear in the container process groups. Recreating the container is required after a `group_add` change.

The serial port is busy

Symptom: the driver reports that it cannot open the port even though it exists and permissions are correct.

First ensure that Terminal A does not already have a live launch. On the host, inspect without killing anything:

```
fuser -v "$LIDAR_DEVICE"
docker compose top
```

Stop the known owner cleanly. Do not start two Unitree drivers and do not kill an unknown process without identifying why it owns the adapter.

RViz2 cannot connect to X11

Symptoms include could not connect to display or a Qt xcb error.

On the host:

```
printf 'DISPLAY=%s\nXAUTHORITY=%s\n' "$DISPLAY" "$XAUTHORITY"
ls -l /tmp/.X11-unix
test -r "$XAUTHORITY"
```

Set the correct DISPLAY or XAUTHORITY, then recreate the container. Do not use `xhost +`.

RViz2 reports an OpenGL or DRI permission error

Recheck the host device groups:

```
export VIDEO_GID="$(stat -Lc %g /dev/dri/card0)"
export RENDER_GID="$(stat -Lc %g /dev/dri/renderD128)"
docker compose down
docker compose up -d
```

For a diagnostic software-rendering attempt, enter Terminal A and set `LIBGL_ALWAYS_SOFTWARE=1` before the launch command. Software rendering may be slow; it does not correct a missing point-cloud stream.

The ROS package is not found

Symptom: Package 'l1_bringup' not found or Package 'l1_octomap_bringup' not found.

Check inside the container:

```
ros2 pkg prefix l1_bringup
ros2 pkg prefix l1_octomap_bringup
ros2 pkg prefix unitree_lidar_ros2
ros2 pkg executables octomap_server
```

If either package is absent, exit the container, rebuild the image, and recreate the service:

```
docker compose down
docker compose build
docker compose up -d
```

Topics exist but no messages arrive

Check in Terminal B:

```
ros2 node list
ros2 topic info /unilidar/cloud
ros2 topic echo /unilidar/cloud --once --field header
```

Then read Terminal A from the first driver message onward. Verify separate sensor power, the adapter and cable, the selected tty, and permission checks. A topic name can exist before useful sensor data arrives.

RViz2 opens but the cloud is blank

First prove that the cloud header arrives with the command above. In RViz2, confirm:

```
Fixed Frame = unilidar_lidar
PointCloud2 topic = /unilidar/cloud
PointCloud2 display = enabled
```

Expand the PointCloud2 display and read its status. Use the mouse to orbit and zoom around the origin. If no message arrives in Terminal B, the fault is upstream of RViz2.

The cloud is visible but OctoMap is empty

Check the data path in Terminal B:

```
ros2 node info /octomap_server
ros2 topic echo /unilidar/cloud --once --field header
ros2 topic echo /octomap_point_cloud_centers --once --field width
```

The cloud frame must be `unilidar_lidar`, OctoMap must subscribe to `/unilidar/cloud`, and measured surfaces must be within the default 15 m map range. Keep the sensor housing fixed and allow several scans. If occupied map data exists but RViz is blank, enable the Occupied OctoMap Voxels display and confirm its topic is `/occupied_cells_vis_array` and the fixed frame is `unilidar_lidar`.

The bag directory already exists

Symptom: `rosbag2` refuses `/workspace/bags/l1_run_01`.

Choose a new name:

```
ros2 bag record -o /workspace/bags/l1_run_02 \  
  /unilidar/cloud /unilidar/imu
```

Do not overwrite or merge recordings accidentally. Use `ros2 bag info` on the old bag and preserve it until its value is known.

The bag has no metadata or zero messages

An abrupt terminal close or `docker compose down` during recording can leave an incomplete bag. Always stop the recorder with `Ctrl-C` first and wait for the prompt. For a repeatable run, use a new output name and confirm both topics before starting.

ENGINEERING NOTE

The shortest reliable fault-isolation order is host tty, container tty permissions, driver output, ROS publisher, one received header, RViz2, then `rosbag2`. This follows the data path and avoids changing unrelated settings.